Design Principles

Evaluating Design Decisions

Software Engineering

FinTech 512

Duke

Where We Left Off …

- Software systems are **collections of decisions**
- Objects are **responsibility holders**
- **Entities vs Policies**
- **HAS-A by default; IS-A only for true specialization**
- Diagrams and reviews focus on **decision visibility**

OO Design Foundations

- **Abstraction** → *make boundaries explicit*
  What others can rely on (interfaces / policy contracts)
- **Encapsulation** → *objects own decisions + invariants*
  Hide internals; expose responsibilities
- **Inheritance** → *IS-A with substitutability*
  If it's for code reuse, reconsider
- **Polymorphism** → *replaceable collaborators*
  Example: Swappable policies (RiskPolicy implementations) without changing the coordinator code

Design principles: The Landscape

General heuristics

- Keep it simple (but no simpler)
- Least surprise/astonishment
- Don't repeat yourself (DRY)
- Low coupling / high cohesion
- Separation of concerns
- **YAGNI** "you aren't gonna need it"

Supporting outcomes

- Design for testability

Core OO principles (SOLID)

- Single Responsibility
- Open / Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Least Surprise/Astonishment

- **No unexpected side-effects**
  - getSomething() should not change state
- **No reliance on hidden knowledge to make it work**
  - "You have to call methodA right before methodB or it crashes"
  - "It got stuck in an infinite loop? Yeah, that happens if you pass a negative number to methodC…"
- **Names make sense**
- **Give constants names –** avoid “magic” numbers

  seconds_in_day = 86400

Don’t Repeat yourself (DRY)

- Same code should only appear **once**
- If code needed more than once, use a function
- **Gut Check**: If you change in place 1 and that requires you to make the same change in place 2, then you’ve violated DRY

Grouping SOLID into Design Pressures

Pressure 1: Responsibility & Cohesion

What this pressure is about

- Assigning clear **ownership** of decisions and behavior
- Grouping related responsibilities that **change together**
- Avoiding classes and functions that try to do “a little of everything”

Principles involved

- Single Responsibility Principle (SRP)
- High Cohesion
- Separation of Concerns

Design questions to ask in reviews

- What decisions live here?
- How many **reasons could this code change**?
- Are multiple concerns mixed together?
- Does this object know *facts*, or does it also decide *policy*?

Common warning signs

- Class descriptions with multiple “ANDs”
- Conditionals selecting between unrelated behaviors
- Entities making policy decisions
- Methods that grow whenever requirements change

Single Responsibility Principle (SRP)

SRP = one reason to change

- A class or function should change for **one kind of decision**

In our terms

- Entities → facts & invariants
- Policies → rules & decisions
- Coordinators → workflows
- Mixing these roles usually violates SRP.

Example: TradeReportGenerator

- Loads trades from storage
- Applies compliance rules
- Formats reports for output

Why this violates SRP

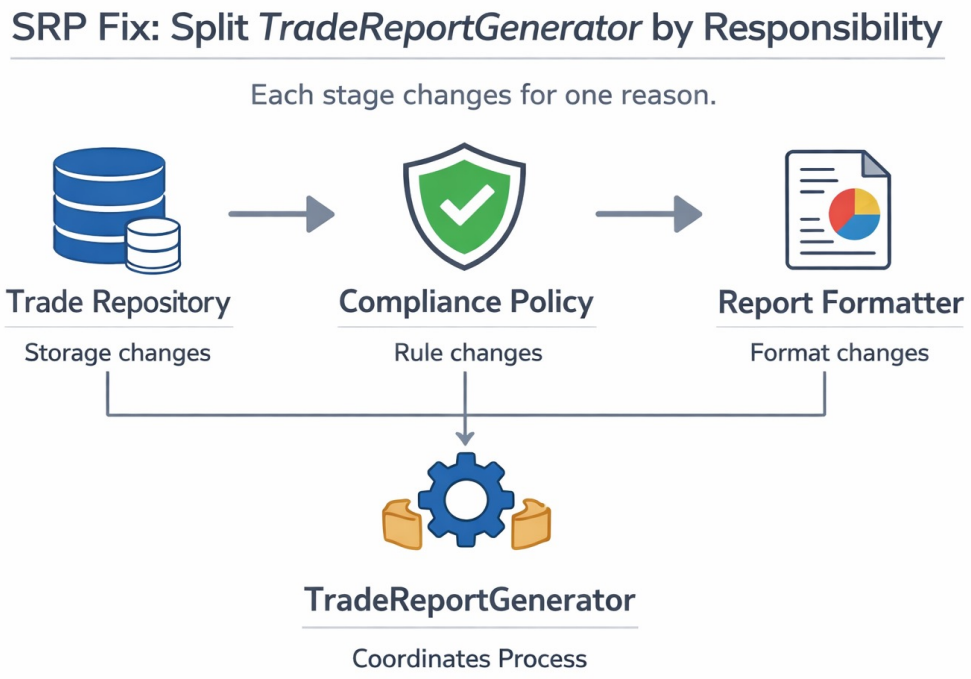
- Data access changes
- Compliance rules change
- Reporting format changes

Three different reasons to change one class

SRP Fix:
Split TradeReportGenerator by Responsibility

Goal: each part changes for **one reason**

- **TradeRepository**
  Gets trades from storage
- **CompliancePolicy**
  Decides what is allowed / flagged
- **ReportFormatter**
  Turns results into PDF/CSV/JSON
- **TradeReportGenerator** *(Coordinator)*
  Orchestrates the workflow; owns no business rules



- Storage changes → repository changes
- Compliance rules change → policy changes
- Output format changes → formatter changes

Pressure 2: Change & Extension

What this pressure is about

- Designing so **new behavior can be added**
- Without rewriting stable code
- And without spreading changes across the system

Principles involved

- Open / Closed Principle (OCP)
- Liskov Substitution Principle (LSP)

Design questions to ask in reviews

- When requirements change, where does the change land?
- Can I add a new rule, policy, or behavior without editing existing logic?
- Are extensions truly substitutable, or do they break assumptions?

Common warning signs

- Large if / else chains selecting behavior
- Business rules scattered across multiple classes
- Adding a new case requires touching many files
- Inheritance used to “almost” fit a new case

Open/Closed Principle (OCP)

The Core Definition:

> "Software entities (classes, modules, functions, etc.) should be **open for extension**, but **closed for modification**."
> — *Bertrand Meyer*

Key Takeaways:

- **Open for extension:** You should be able to make the module behave in new and different ways as requirements change.
- **Closed for modification:** You should never have to change the source code of a module that is already tried, tested, and working.

The "Anti-Pattern" (Violating OCP)

```
class PaymentProcessor:
    def process_payment(self, payment_type, amount):
        if payment_type == "credit_card":
            print(f"Processing ${amount} via Stripe API.")
        elif payment_type == "paypal":
            print(f"Processing ${amount} via PayPal API.")

        # To add 'crypto', we must EDIT this class and add another 'elif'.
        # This violates "Closed for Modification."
```

The Solution

Use an **Abstract Base Class (ABC)** to define a "contract" that all payment methods must follow.

The Strategy:

- **Define a Base:** Create an abstract class PaymentMethod.
- **Extend:** Create specific classes (e.g., CreditCardPayment, CryptoPayment) that implement the base.
- **Execute:** The processor interacts only with the abstract base.

The Refactored Python Code

Step 1: The Abstract Base

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def authorize(self, amount):
        pass

    @abstractmethod
    def execute(self, amount):
        pass
```

Step 2: Extensions (Adding New Features)

```python
class CryptoPayment(PaymentMethod):
  def authorize(self, amount):
    print("Validating wallet signature...")

  def execute(self, amount):
    print(f"Transferring {amount} BTC on the blockchain.")


class ApplePay(PaymentMethod):
  def authorize(self, amount):
    print("Validating FaceID/TouchID...")

  def execute(self, amount):
     print(f"Charging ${amount} via Apple Pay.")
```

The Stable Core (Closed for Modification)

```python
class PaymentManager:
    def process(self, method: PaymentMethod, amount: float):
        # This code never changes!
        method.authorize(amount)
        method.execute(amount)


# Usage:
manager = PaymentManager()
manager.process(CryptoPayment(), 500.00)
manager.process(ApplePay(), 25.50)
```

PaymentManager is completely decoupled from the specific payment types..

Why This Matters for FinTech

- **Security:** You can lock down your "Core Accounting" code. Since you don't modify it to add features, the risk of introducing security vulnerabilities into proven code is minimized.
- **Vendor Agility:** If you want to switch from Stripe to Adyen, you just create a new `AdyenPayment` class. The rest of your app remains untouched.
- **Testability:** You can write unit tests for `CryptoPayment` in isolation without needing to mock the entire `PaymentProcessor`.

Liskov Substitution Principle (LSP)

The Core Definition:

> Objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program.
>
> — *Barbara Liskov*

In Plain English:

- If you have a function that processes a generic **Account**, it should be able to process a **SavingsAccount** or a **CheckingAccount** without knowing the difference and without crashing.
- Subclasses must honor the "contract" established by the parent class.

The FinTech Violation (The "Broken" Hierarchy)

Imagine we have a base class for all bank accounts. We assume all accounts allow withdrawals.

The Problem: We introduce a `FixedDepositAccount` where money is locked for a year.

```
class Account:
    def withdraw(self, amount):
        print(f"Withdrawing ${amount}")

class CheckingAccount(Account):
    def withdraw(self, amount):
        print(f"Withdrawing ${amount} from Checking")

class FixedDepositAccount(Account):
     def withdraw(self, amount):
         # Violation: This account type doesn't allow withdrawals!
         raise Exception("Withdrawals not allowed on Fixed Deposits!")
```

The Crash Scenario: The following code is fragile and unpredictable

```
def process_emergency_funds(accounts: list[Account]):
    for account in accounts:
        # This will crash the entire loop when it hits a FixedDepositAccount
        account.withdraw(100)
```

Correct Implementation

Modify the inheritance hierarchy

```
class Account:
    def __init__(self, balance):
        self.balance = balance

class WithdrawableAccount(Account):
    def withdraw(self, amount):
        self.balance -= amount
        print(f"Withdrawn: {amount}")

class CheckingAccount(WithdrawableAccount):
    pass # Inherits withdraw behavior correctly

class FixedDepositAccount(Account):
    def generate_interest(self):
        self.balance *= 1.05 # Unique behavior
```

Why This Matters for FinTech

- **Don't force behaviors:** If a subclass can't fulfill a method (like `withdraw()`), that method shouldn't be in the base class.
- **The "Tell, Don't Ask" Rule:** You shouldn't have to use `isinstance(account, FixedDeposit)` to check if a method is safe to call.
- **Predictability:** LSP ensures that as your banking app grows, adding a "High Yield Savings Account" won't accidentally break your ATM withdrawal logic.

Pressure 3: Dependencies & Direction

What this pressure is about

- Controlling **what depends on what**
- Preventing low-level details from shaping high-level decisions
- Making dependencies **replaceable and reviewable**

Principles involved

- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)
- Dependency Injection (DI)
- Design for Testability

Design questions to ask in reviews

- What does this class **depend on directly**?
- Could I **replace this dependency** without rewriting logic?
- Does this object know **too much about others' internals**?
- Is dependency direction aligned with **decision ownership**?

Common warning signs

- Business logic directly creates concrete dependencies
- Large, "do-everything" interfaces
- Code that is hard to test without building the whole system

Interface Segregation Principle (ISP)

The Core Definition:

> Clients should not be forced to depend upon interfaces that they do not use.
>
> — *Robert C. Martin*

In Plain English:

- It is better to have many **specific** interfaces than one "fat" general-purpose interface.
- In Python, this means we should avoid creating large Abstract Base Classes (ABCs) that force subclasses to implement methods they don't actually need.

The “Fat” Interface

```python
from abc import ABC, abstractmethod

class BankService(ABC):
  @abstractmethod
  def withdraw(self): pass

  @abstractmethod
  def deposit(self): pass

  @abstractmethod
  def generate_regulatory_report(self): pass

  @abstractmethod
  def internal_audit(self): pass

class ATM(BankService):
  def withdraw(self):
    print("Dispensing cash...")

  def deposit(self):
    print("Accepting envelope...")

  def generate_regulatory_report(self):
    # Violation: An ATM should not have access to or need to implement this!
   raise NotImplementedError("ATMs cannot generate reports.")

  def internal_audit(self):
    # Violation: Security risk! Why does an ATM have audit logic?
    raise NotImplementedError("ATMs cannot perform audits.")
```

Why "Fat" Interfaces are Dangerous

- **Security Risk:** By providing an `internal_audit` method to an ATM client, you've exposed an entry point that shouldn't exist in that context.
- **Fragility:** If you change the parameters for `generate_regulatory_report`, you have to update the ATM class, even though the ATM doesn't use that feature.
- **Cognitive Load:** Developers have to navigate dozens of irrelevant methods when trying to implement a simple feature.

The Solution (Splitting the Interfaces)

Break the "Fat" interface into smaller, logically grouped "Role-Based" interfaces.

The Strategy:

- **Transactional Interface:** withdraw, deposit.
- **Reporting Interface:** generate_regulatory_report.
- **Audit Interface:** internal_audit.

Dependency Inversion Principle (DIP)

The Core Definition:

1. High-level modules should not depend on low-level modules. Both should depend on **abstractions**.
2. Abstractions should not depend on details. **Details** (concrete implementations) should depend on **abstractions**.

In Plain English:

- Your "Gold" business logic (e.g., how you calculate interest) shouldn't be "stuck" to your "Concrete" infrastructure (e.g., a specific Oracle database or a specific SMS provider).
- You should depend on a *concept* (an interface), not a *specific tool*.

The Dependency Inversion Violation

```python
class TwilioSmsService:
    def send_sms(self, phone, message):
        print(f"Sending Twilio SMS to {phone}: {message}")

class TransactionMonitor:
    def __init__(self):
        # Violation: High-level module depends directly on low-level Twilio service
        self.sms_service = TwilioSmsService()

    def check_transaction(self, amount):
        if amount > 10000:
            self.sms_service.send_sms("555-0199", "Large transaction detected!")
```

- **Rigidity:** Switching providers (e.g., Twilio to SendGrid) requires rewriting core logic.
- **Untestable:** Testing forces live API calls or complex, fragile "monkey-patching."
- **Lock-in:** Core business rules are tethered to specific databases or third-party libraries.

The Solution (Inverting the Dependency)

Strategy:

- Insert an **abstraction** between logic and infrastructure.
- **Invert control:** Low-level services must conform to high-level interfaces.

The Approach:

- **Define the Interface:** `NotificationProvider` (the abstraction).
- **Inject the Dependency:** Pass the specific provider into the monitor at runtime.

Refactored Solution

```python
from abc import ABC, abstractmethod

# 1. The Abstraction
class NotificationProvider(ABC):
    @abstractmethod
    def send(self, recipient, message): pass

# 2. Low-level Details (implementing the abstraction)
class TwilioSms(NotificationProvider):
    def send(self, recipient, message):
        print(f"SMS via Twilio: {message}")

class EmailAlert(NotificationProvider):
    def send(self, recipient, message):
        print(f"Email via SendGrid: {message}")

# 3. High-level Module (depends ONLY on the abstraction)
class TransactionMonitor:
    def __init__(self, notifier: NotificationProvider):
        self.notifier = notifier

def check_transaction(self, amount):
    if amount > 10000:
        self.notifier.send("user@example.com", "High-value alert!")
```

Dependency Inversion Benefits

- **Easy Testing:** Use **mocks** to verify logic without hitting live APIs.
- **Plug-and-Play:** Swap databases or gateways (e.g., Stripe to Adyen) via **adapters**.
- **Parallel Development:** Teams build core logic and integrations simultaneously using **shared interfaces**.

Designing for testing

- **Controllability**: how well can my test code control the state of the module I want to test?
- **Observability**: how well can my test code inspect the state (to see if correct) of the module I want to test?
- **Isolatability**: how well can I separate out the module I want to test it by itself?

Kent Beck's Four Rules of Software Design

- "Passes the tests"
- "Reveals intention" – code is easy to understand
- "No duplication" (aka the DRY principle – don't repeat yourself)
- "Fewest elements" – if it's not needed, don't include it.

  - https://martinfowler.com/bliki/BeckDesignRules.html